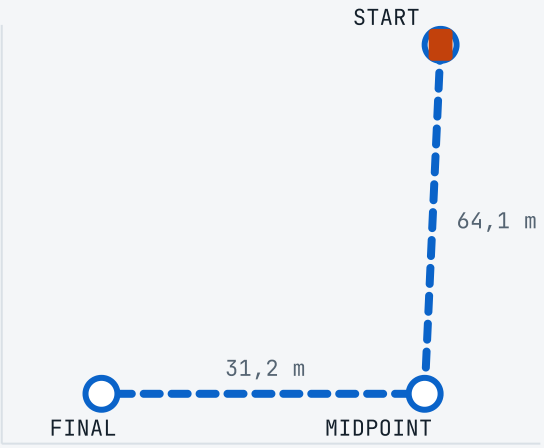
GUIA DE INSTALAÇÃO E USO · CIN-UFPE

Fleet UI para o teste BLE

Simulação aberta do percurso, com um botão para executar e os scripts de análise já integrados. Este guia leva do computador vazio até o gráfico de RSSI.



96,3 m	0,248 m/s	≈ 6,5 min	6 de 6
percorridos no replay pela tela	velocidade média (artigo: 0,244 a 0,253)	por corrida completa	corridas reais da Tabela 3 recalculadas

0

-60 dBm

Antes de começar

- Ubuntu 22.04 serve. Não é preciso instalar ROS no computador: tudo roda dentro de um contêiner com Ubuntu 24.04 e ROS 2 Jazzy.
- Uns 15 GB livres em disco e internet boa na primeira vez (o download é grande).
- Acesso ao repositório no GitHub.

1

-75 dBm

Instalar o Docker

Use o Docker oficial. Os pacotes antigos `docker.io` e `docker-compose` do Ubuntu não servem.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/key
rings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/doc
ker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && ech
o "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.lis
t > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
sudo usermod -aG docker $USER
```

> **Depois do último comando, saiam da sessão e entrem de novo** (ou reiniciem). Para conferir, `docker run hello-world` deve funcionar sem `sudo`.

2

-82 dBm

Baixar e montar

```
git clone https://github.com/eduardowanderleyde/fleet-ui-ufpe.git ~/fleet-ui
cd ~/fleet-ui
make docker-build   # só na primeira vez: 15 a 30 min
```

Se o repositório for privado, o GitHub pede login: usem `gh auth login` ou um token de acesso, porque a senha da conta não funciona no `git clone`.

> **Sem GitHub:** se o Eduardo entregou o arquivo `fleet-ui-ufpe-….zip`, descompactem e entrem na pasta. O resto é igual:
>
> ```
> unzip fleet-ui-ufpe-*.zip -d ~
> cd ~/fleet-ui
> make docker-build
> ```

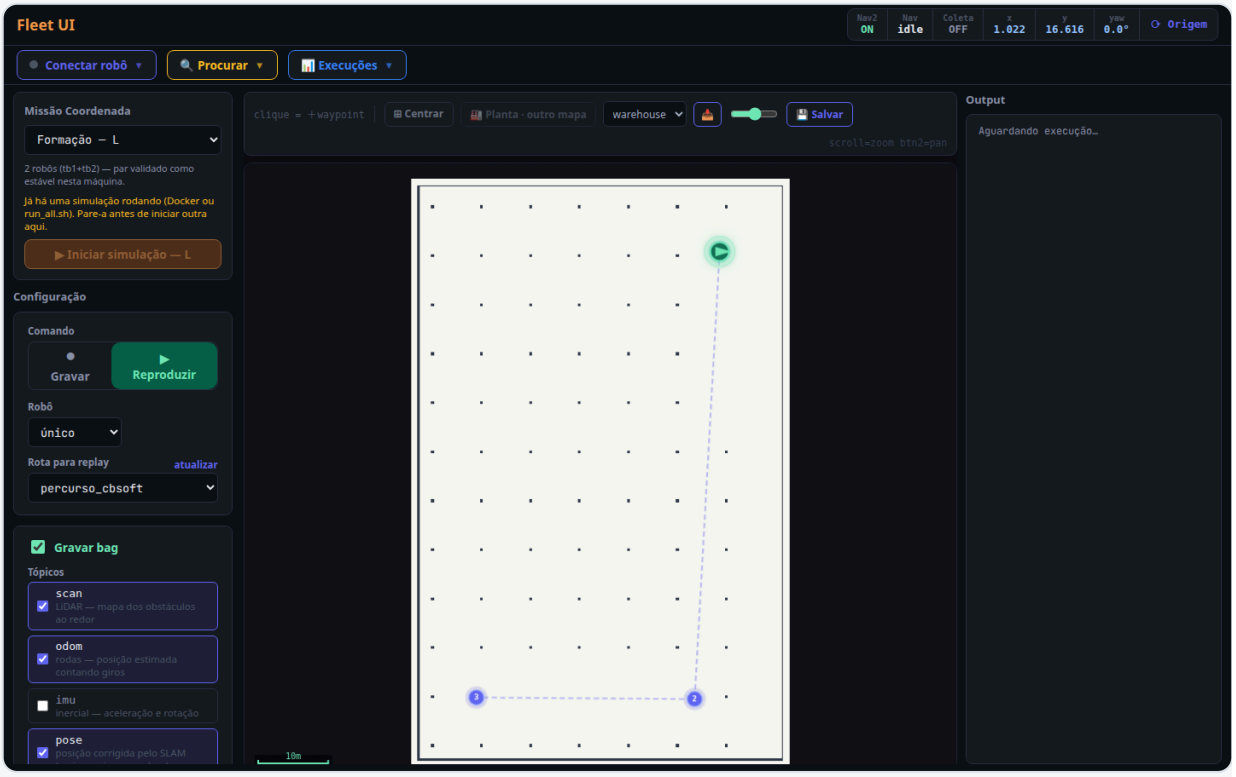
3 Ligar a simulação

-88 dBm

```
cd ~/fleet-ui
make cbsoft-up
```

Esse terminal fica ocupado mostrando o que está acontecendo; deixem aberto. Em cerca de 1 minuto, abram `http://localhost:5173` no navegador.

O robô aparece no START de vocês, num salão aberto com coordenadas iguais às do mapa `hall_cin_v4`. A simulação usa o domínio ROS 42, para nunca conversar com um TurtleBot4 de verdade que esteja na mesma rede (eles usam o domínio 0).

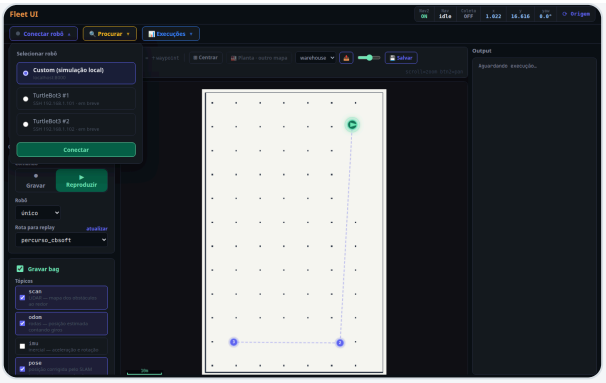


A tela já abre em Reproduzir, com o percurso `percurso_cbsoft` desenhado.

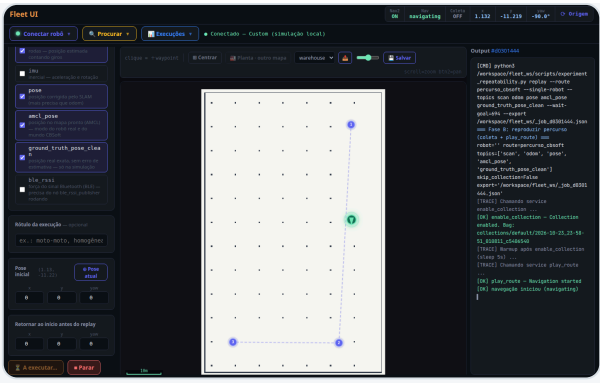
4 Rodar o percurso pela tela

-93 dBm

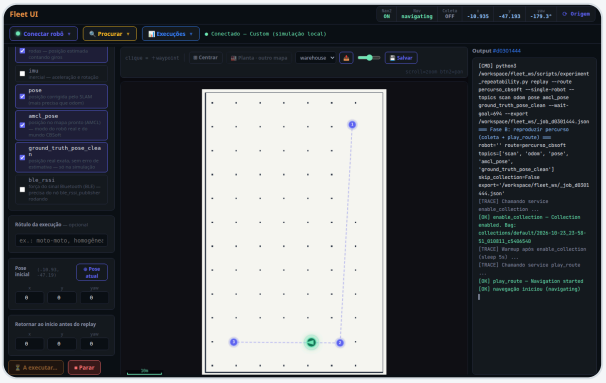
1. Cliquem em **Conectar robô**, deixem marcado **Custom (simulação local)** e cliquem em **Conectar**. Isso só confirma que a tela está falando com o servidor; não procura Bluetooth. As opções "TurtleBot3 · em breve" ainda não funcionam.
2. Cliquem em **▶ Executar replay**. O robô percorre os 96 m em cerca de 6,5 minutos, gravando os sensores.
3. No fim, o painel da direita mostra **Sucesso**, a distância e a velocidade média.



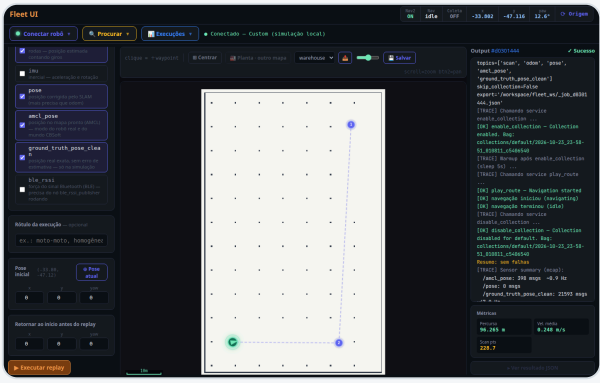
Conectar: deixem “Custom (simulação local)”.



Primeiro trecho, os 64 m em linha reta.



Depois da curva, os 31 m finais.



Fim: distância e velocidade média no canto direito.

> **Para repetir uma corrida,** parem o terminal do passo 3 com Ctrl+C e rodem `make cbsoft-up` de novo (cerca de 1 min). O robô volta ao START. A volta automática pela tela existe, mas ainda não foi validada numa rota deste tamanho.

5

-97 dBm

Ou rodar o script original de vocês

Com a simulação ligada, num segundo terminal:

```
cd ~/fleet-ui
make cbsoft-run
```

Roda o `nav_3points.py` sem nenhuma alteração de coordenadas e grava o CSV de sempre em `fleet_ws/collections/cbsoft/`. Em duas corridas assim, o robô andou 96,3 e 96,5 m a 0,254 e 0,253 m/s, em 382 e 383 s.

6

-101 dBm

Gerar o gráfico de RSSI

Os scripts de vocês (`sync_rssi_nav.py` e `analise_agregados.py`) estão em `fleet_ws/scripts/cbsoft/analise/`. O replay pela tela grava em MCAP; primeiro,

convertam para o mesmo CSV do `nav_3points.py`:

```
make cbsoft-csv   # pega a gravação mais recente
```

Depois, coloquem o log do nRF Connect dentro de `fleet_ws/collections/` e rodem:

```
make cbsoft-analise NAV=collections/cbsoft/<navegacao>.csv NRF="collections/
cbsoft/nrf/<log>.txt"
```

O PNG sai ao lado do CSV. Os horários do CSV usam o fuso do computador, para casar com o log do celular.

Conferimos com os dados de 19/05

Recalculando a Tabela 3 do artigo a partir dos arquivos brutos das seis corridas, com a sincronização de vocês:

Config.	Corrida	RSSI artigo	RSSI refeito	Dist. artigo	Dist. refeita
Homogênea	1	−89,83	−89,72	68,4 m	68,6 m
Homogênea	2	−90,51	−90,35	67,3 m	67,6 m
Homogênea	3	−91,97	−91,70	66,9 m	67,2 m
Heterogênea	1	−95,72	−95,36	39,4 m	39,2 m
Heterogênea	2	−97,30	−97,16	47,3 m	47,3 m
Heterogênea	3	−97,11	−96,74	41,6 m	41,7 m

As diferenças ficam abaixo de 0,4 dBm e 0,3 m.

!

-103 dBm

Um detalhe no script de vocês

O `nav_3points.py` assina `/amcl_pose` com o QoS padrão (VOLATILE). O AMCL publica essa pose como TRANSIENT_LOCAL e só republica quando o robô se move. Por isso, as primeiras linhas do CSV podem sair com pose (0, 0).

Aconteceu numa das seis corridas: a homogênea 1, de 19/05 às 15:57, com 3 linhas em (0, 0). Descartando essas linhas, a distância até desconectar dá 68,6 m, contra 68,4 m no artigo. **O resultado publicado não foi afetado.**

Para as próximas corridas, assinar com a mesma durabilidade do AMCL resolve:

```
from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy

amcl_qos = QoSProfile(depth=1,
                      durability=DurabilityPolicy.TRANSIENT_LOCAL,
                      reliability=ReliabilityPolicy.RELIABLE)
self.create_subscription(PoseWithCovarianceStamped, "/amcl_pose",
                         self.pose_callback, amcl_qos)
```

7

desconectou

No dia do teste com o TurtleBot4

A simulação está pronta. O que falta só dá para resolver no laboratório, com o robô de verdade:

PRONTO

Percurso e análise

Percurso reproduzido, botão de execução, conversão para CSV e scripts de análise de vocês funcionando.

A TESTAR

Instalação na máquina de vocês

Docker no Ubuntu 22.04 e o primeiro `make docker-build`.

A TESTAR NO ROBÔ

Jazzy conversando com Humble

As actions do Nav2 mudaram entre as versões, então o Nav2 roda no PC, do lado do Fleet UI, como no setup de vocês. O robô só manda laser, odometria e TF e recebe velocidade; essas mensagens são idênticas no Humble e no Jazzy.

A FAZER

Leitura BLE no robô

A leitura do Bluetooth a bordo do robô real ainda não está integrada ao Fleet UI.

No dia: ligar e conectar pela tela

Com o PC na rede do robô (a **TurtleLab**, sem internet) e o Nav2 de vocês desligado (ele é Humble e não conversa com o do Fleet UI):

```
cd ~/fleet-ui
make robot-up ROBOT_IP=192.168.50.11   # o IP do robô, não o do roteador
```

Isso liga só a tela, em `http://localhost:5173`, já no domínio e middleware do robô (0 e CycloneDDS). No painel **Robô real**, à esquerda, ▶ **Conectar** faz em sequência o que antes era do terminal: confere se o IP responde, roda a checagem pré-voo e, sem nenhuma FALHA, sobe no PC o mapa `hall_cin_v4`, o AMCL com o robô no START e o Nav2.

A checagem só escuta, não publica nada. Confere os nós do robô, os tópicos `/scan`, `/odom`, `/tf` e `/tf_static`, as taxas, o TF `odom → base_link` e a defasagem de relógio entre robô e PC, os mesmos problemas do guia de troubleshooting de vocês, e avisa se já há um Nav2 rodando. Cada linha aparece na tela com ✓ ou ✗. ↻ **Reconectar** desliga tudo e refaz, por exemplo depois de acertar o relógio ou trocar o IP. Ao conectar, o robô gira uns 10° para um lado e volta, no lugar: é o sinal de que obedece ao PC (`ROBOT_CONNECT_WIGGLE=false` desliga).

Testamos tudo isso com um robô falso, que publica laser, odometria e TF como o TurtleBot4: IP errado recusado, checagem OK, Nav2 ativo em cerca de 25 s, deslocamento de 1 m concluído e percurso gravando a posição. O teste com o robô de verdade é o que falta. Se o painel não conectar, o jeito antigo continua: `make robot-check` e `make robot-up-terminal`.

> **Segurança.** Na primeira vez, deixem o robô com espaço livre em volta e alguém perto dele. O botão ■ **Parar** da tela cancela a navegação, mas leva cerca de 3 s para o robô parar (ele anda ainda uns 80 cm): numa emergência, a parada é física. Comecem com um deslocamento curto antes do percurso de 96 m. O roteiro do dia tem o passo a passo completo.

?

ajuda

Se algo der errado

“permission denied” ao rodar docker

O usuário ainda não está no grupo `docker`. Saiam da sessão e entrem de novo depois do `usermod` do passo 1.

O terminal repete “Network is unreachable” sem parar

A rede do notebook mudou com a simulação ligada (por exemplo, trocar para a TurtleLab, que não tem internet). Com o `git pull` desta versão o Gazebo não reclama mais, mas o ROS ainda perde a comunicação quando a rede muda. Conectem o notebook na rede em que vão ficar **antes** de ligar a simulação; se ela mudar no meio, parem com Ctrl+C e rodem:

```
make docker-down
make cbsoft-up
```

O terminal repete “list of world names” sem parar

O servidor do Gazebo caiu na subida. Parem com Ctrl+C e rodem:

```
make docker-down
make cbsoft-up
```

A página em localhost:5173 não abre

Esperem o terminal do passo 3 mostrar o servidor pronto (cerca de 1 min). Se a porta estiver ocupada por outro programa, fechem esse programa e subam de novo.

O robô parou no meio ou o Nav2 abortou

Reiniciem com Ctrl+C e `make cbsoft-up`. Se acontecer sempre, mandem para o Eduardo a saída de `make docker-logs`.

Alguém da rede também usa o domínio 42

Escolham outro número:

```
make cbsoft-up CBSOFT_DOMAIN=57
```

Fleet UI, repositório `fleet-ui-ufpe`. Experimento original: “Wireless Connectivity Testing with Mobile Robots: A Bluetooth Low Energy Case Study”, CBSoft 2026.

Dúvidas: falem com o Eduardo.